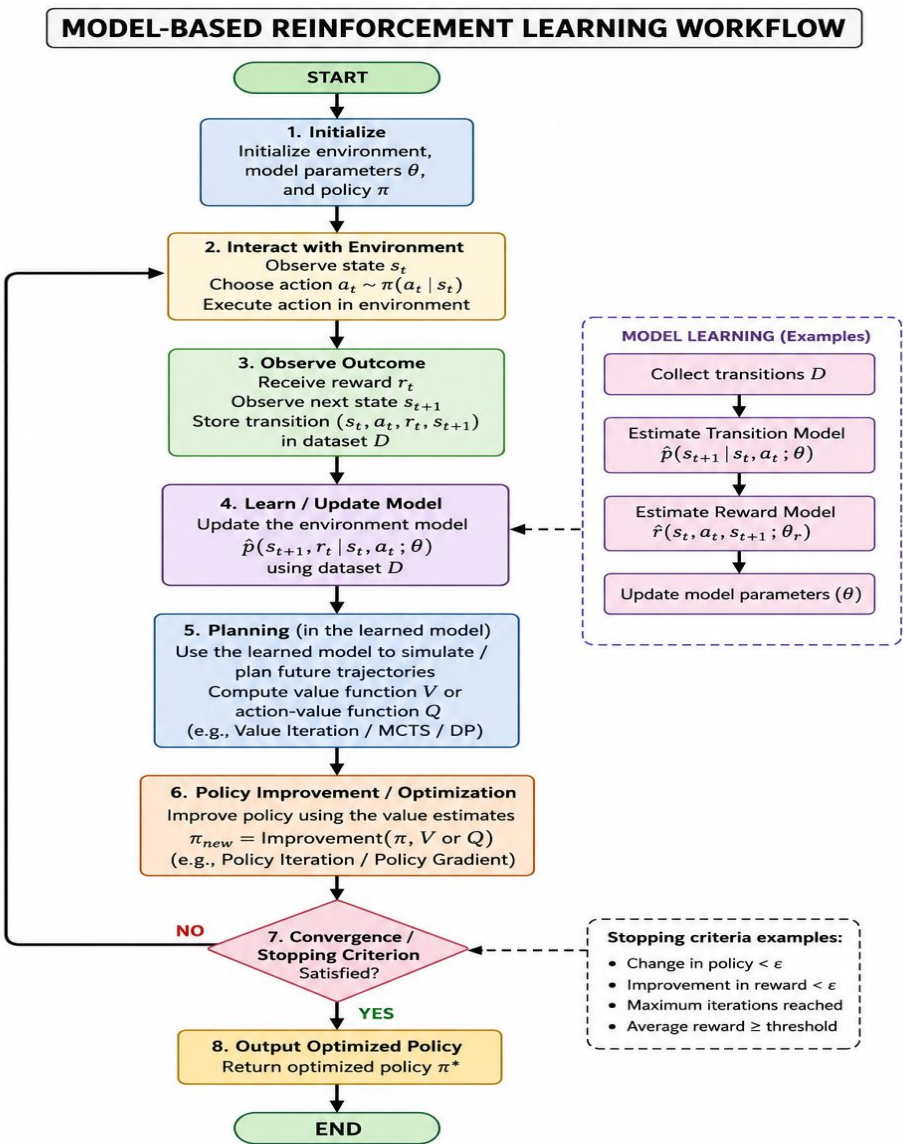
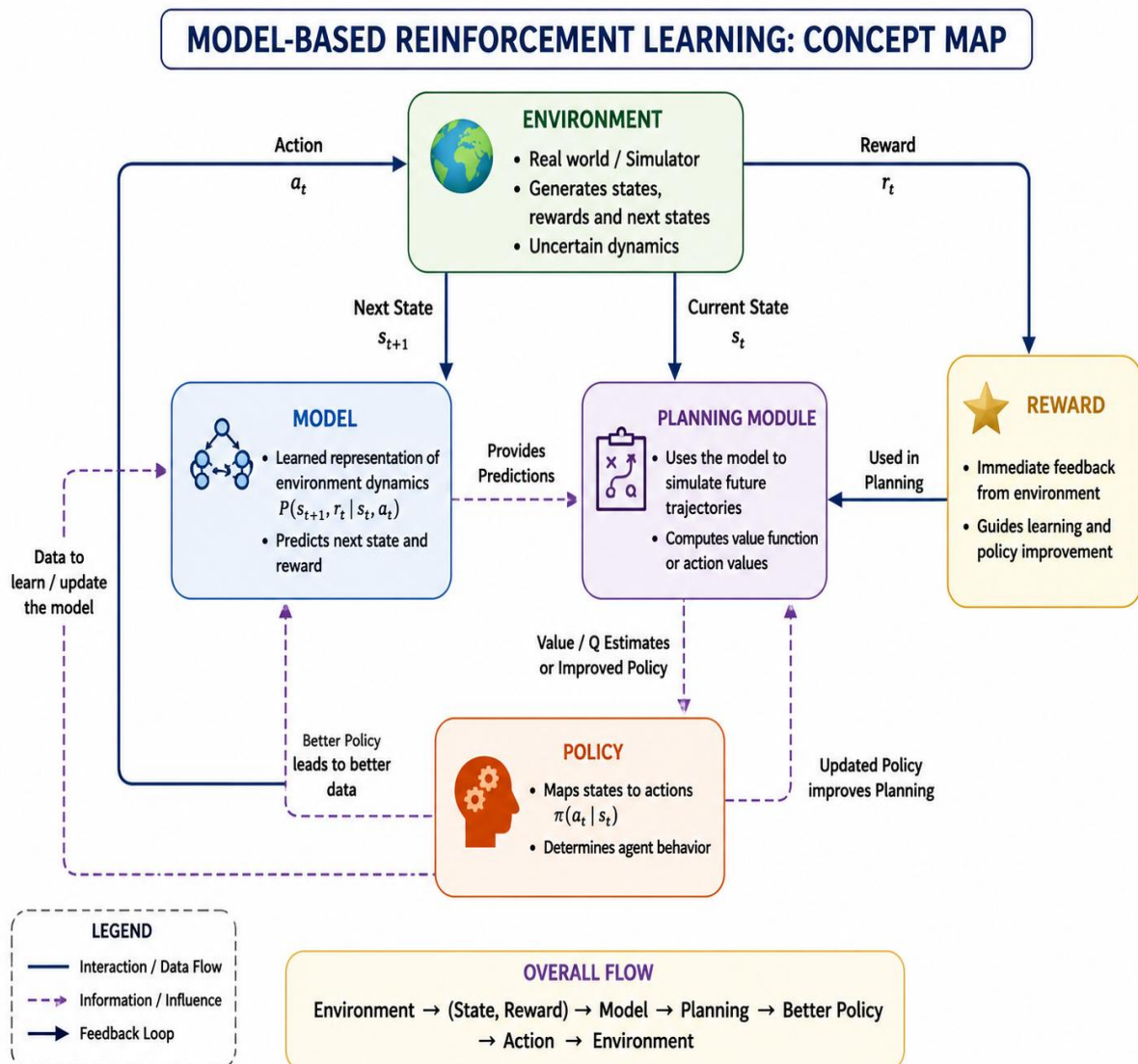


CO4-AT2

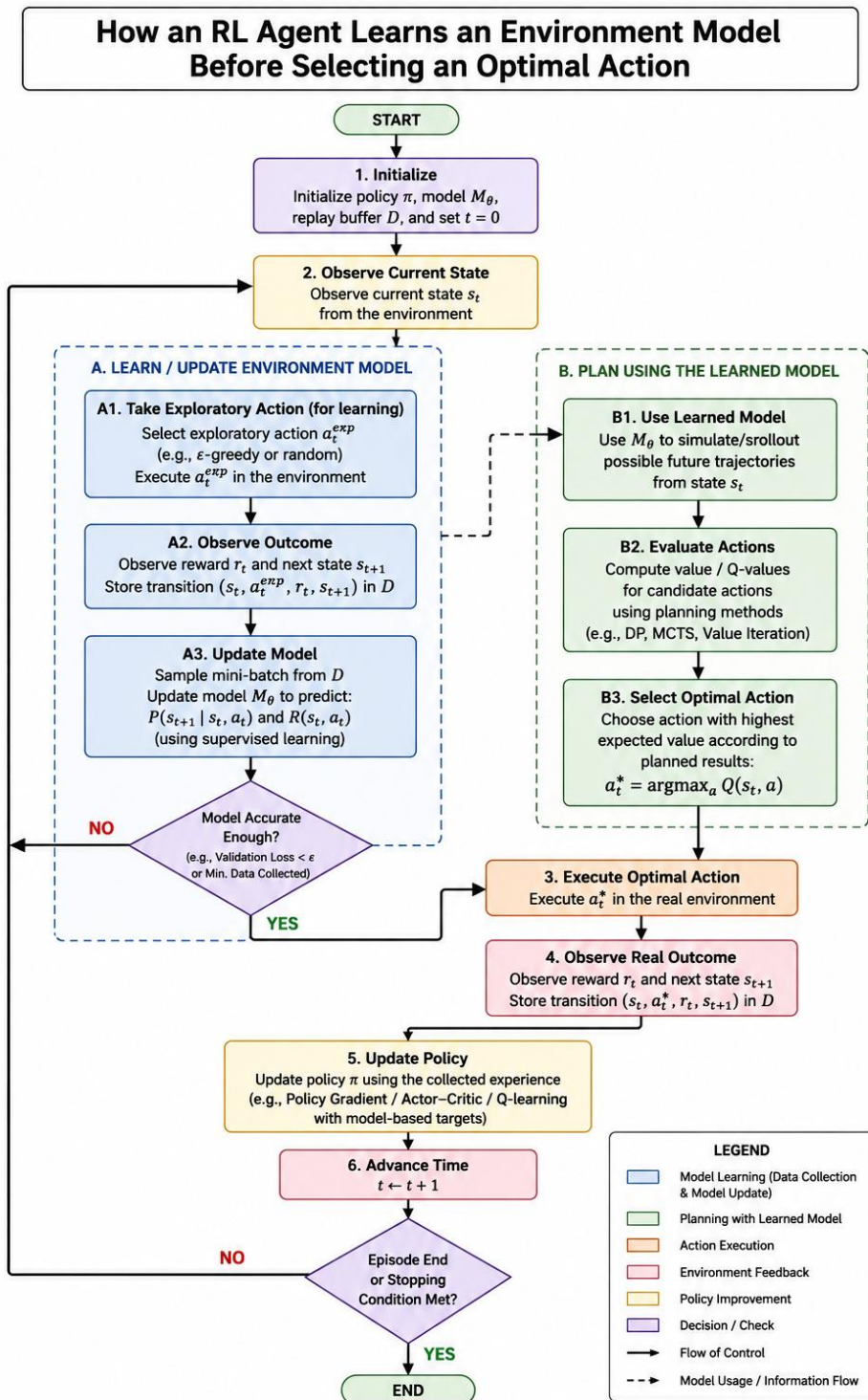
1) Draw a flowchart illustrating the complete workflow of a Model-Based Reinforcement Learning agent from environment interaction to policy optimization.



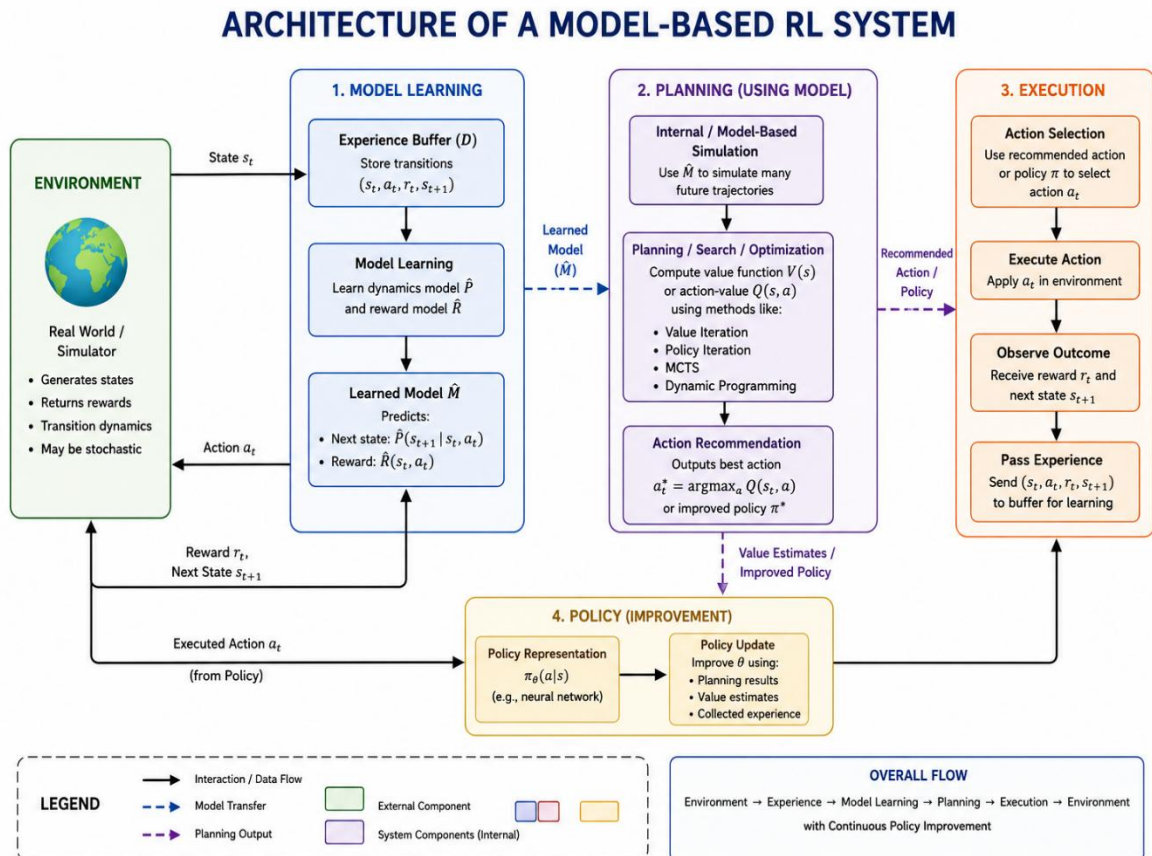
2) Draw a concept map showing the relationship between the Environment, Model, Planning Module, Policy, and Reward in Model-Based Reinforcement Learning.



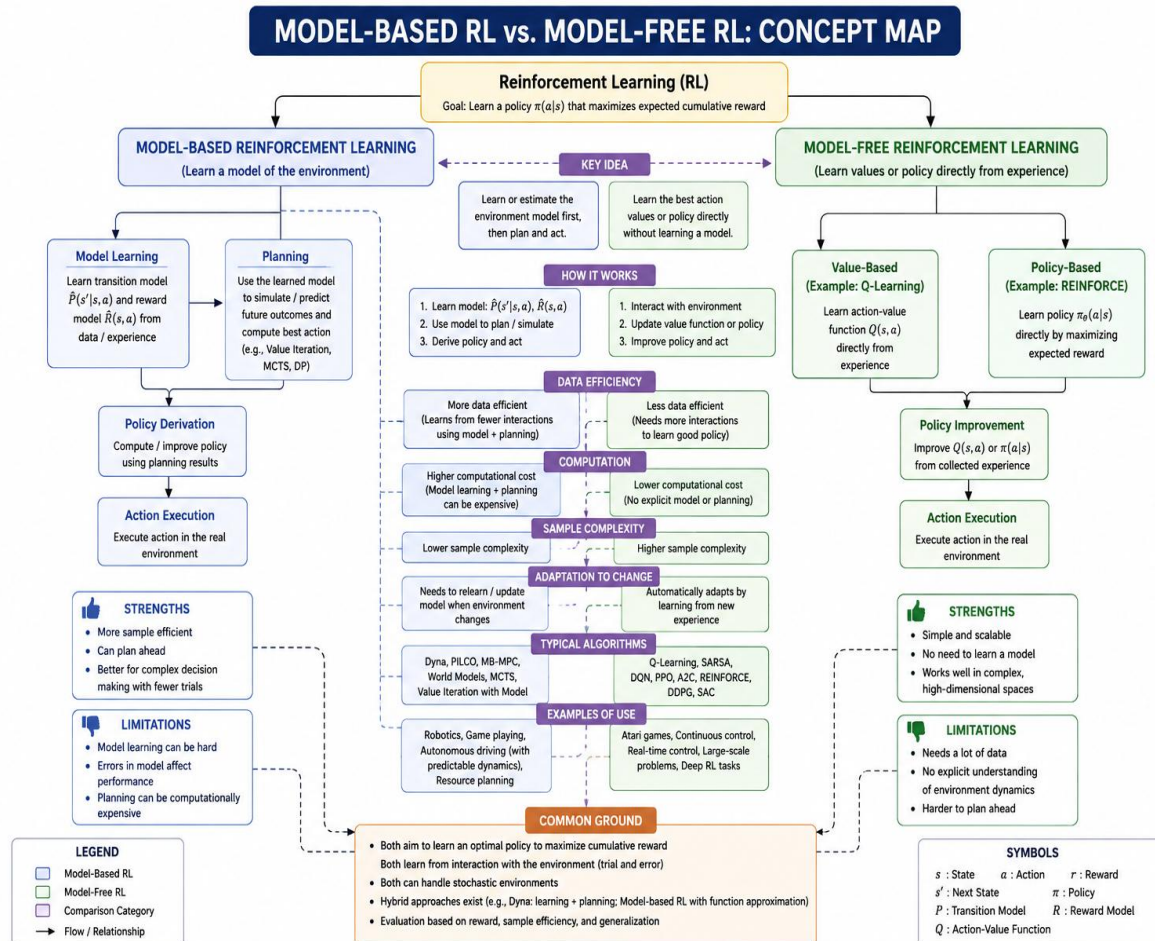
3) Design a flowchart explaining how an RL agent learns an environment model before selecting an optimal action.



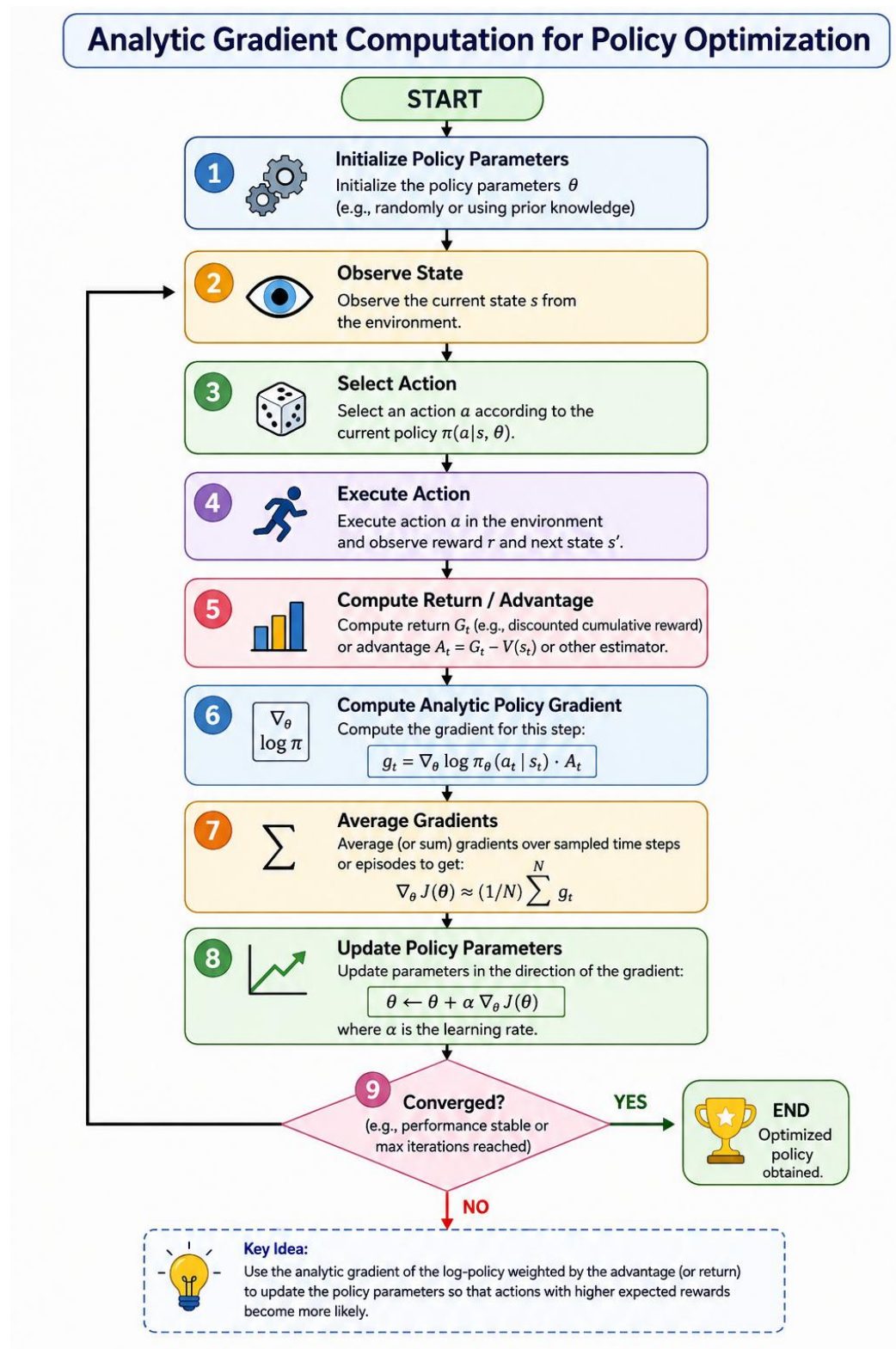
4) Draw the architecture of a Model-Based RL system showing model learning, planning, and execution components.



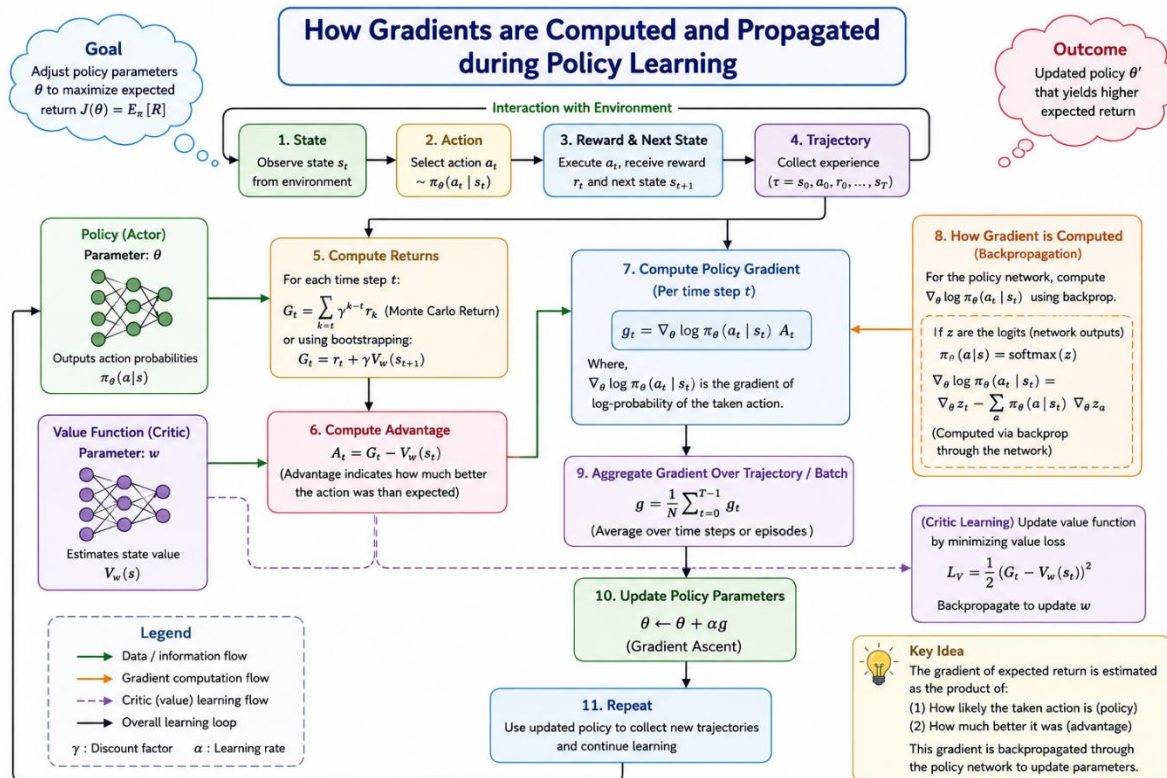
5) Prepare a concept map comparing Model-Based Reinforcement Learning and Model-Free Reinforcement Learning.



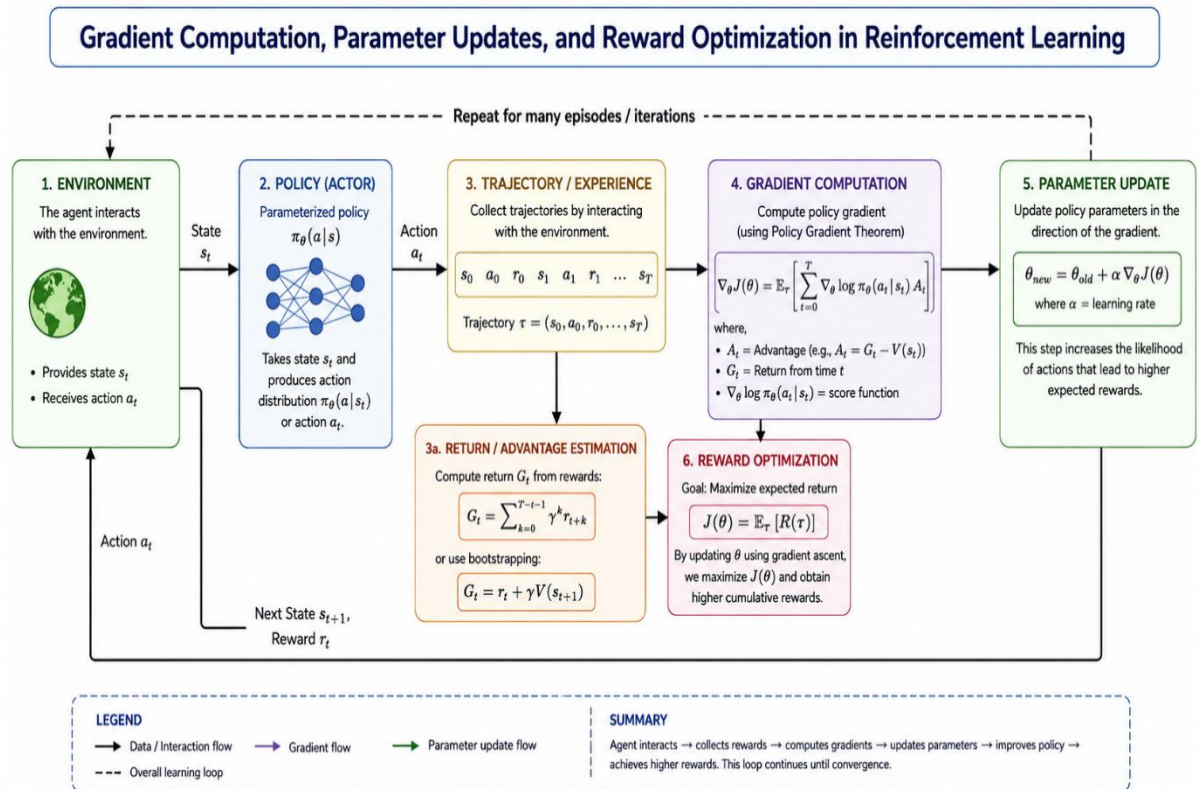
6) Draw a flowchart explaining the steps involved in analytic gradient computation for policy optimization.



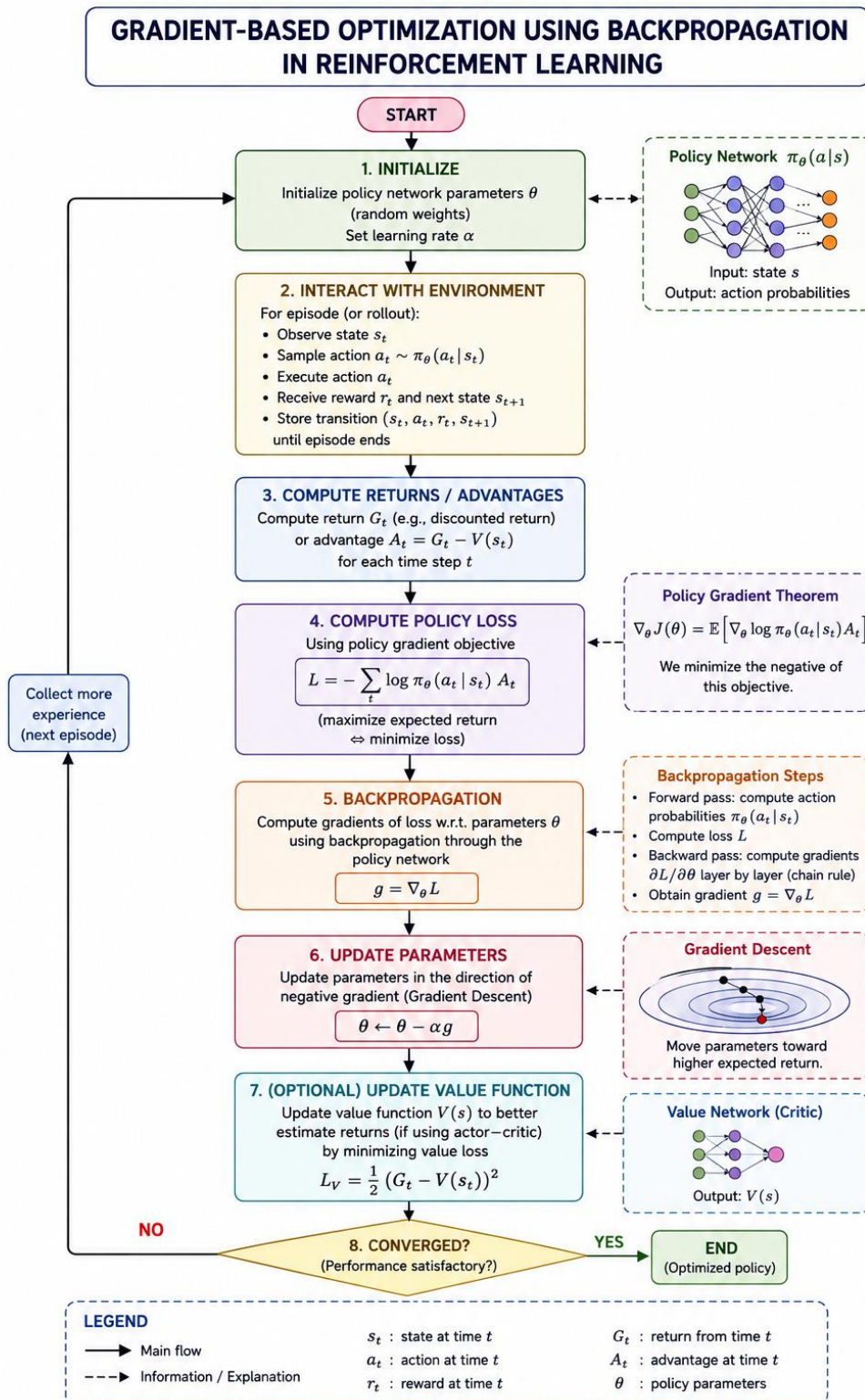
7) Design a concept map showing how gradients are computed and propagated during policy learning.



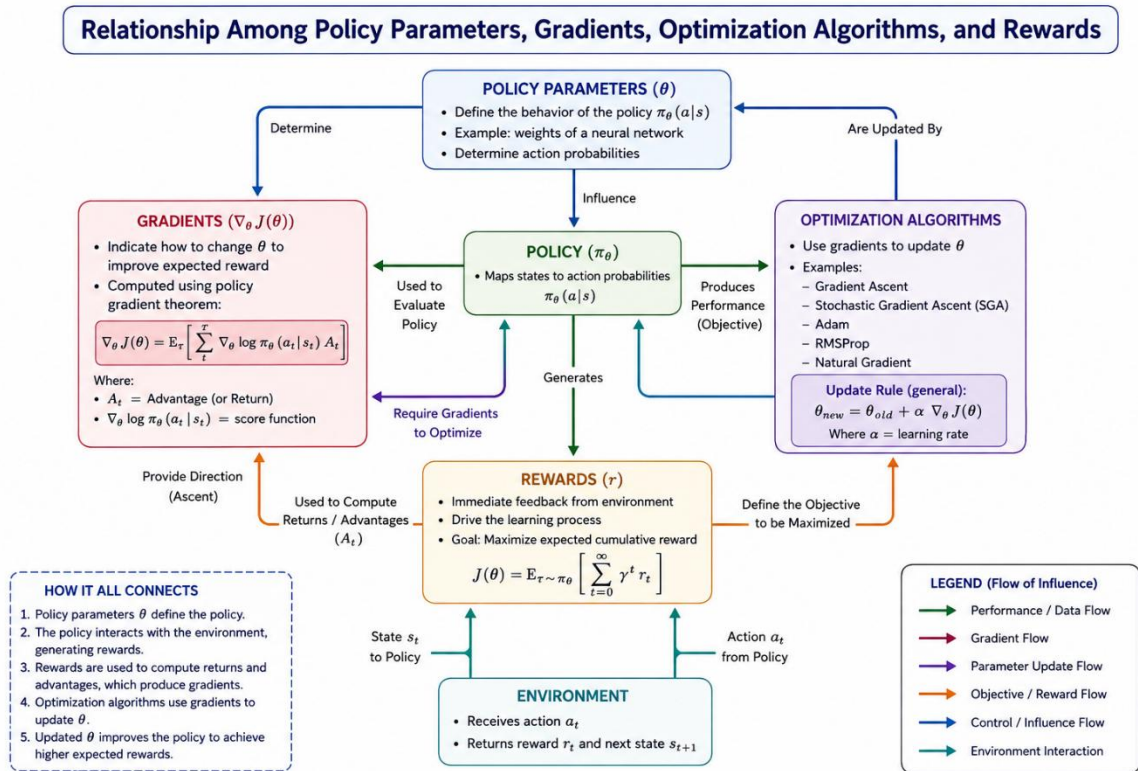
8) Draw a block diagram illustrating gradient computation, parameter updates, and reward optimization in Reinforcement Learning.



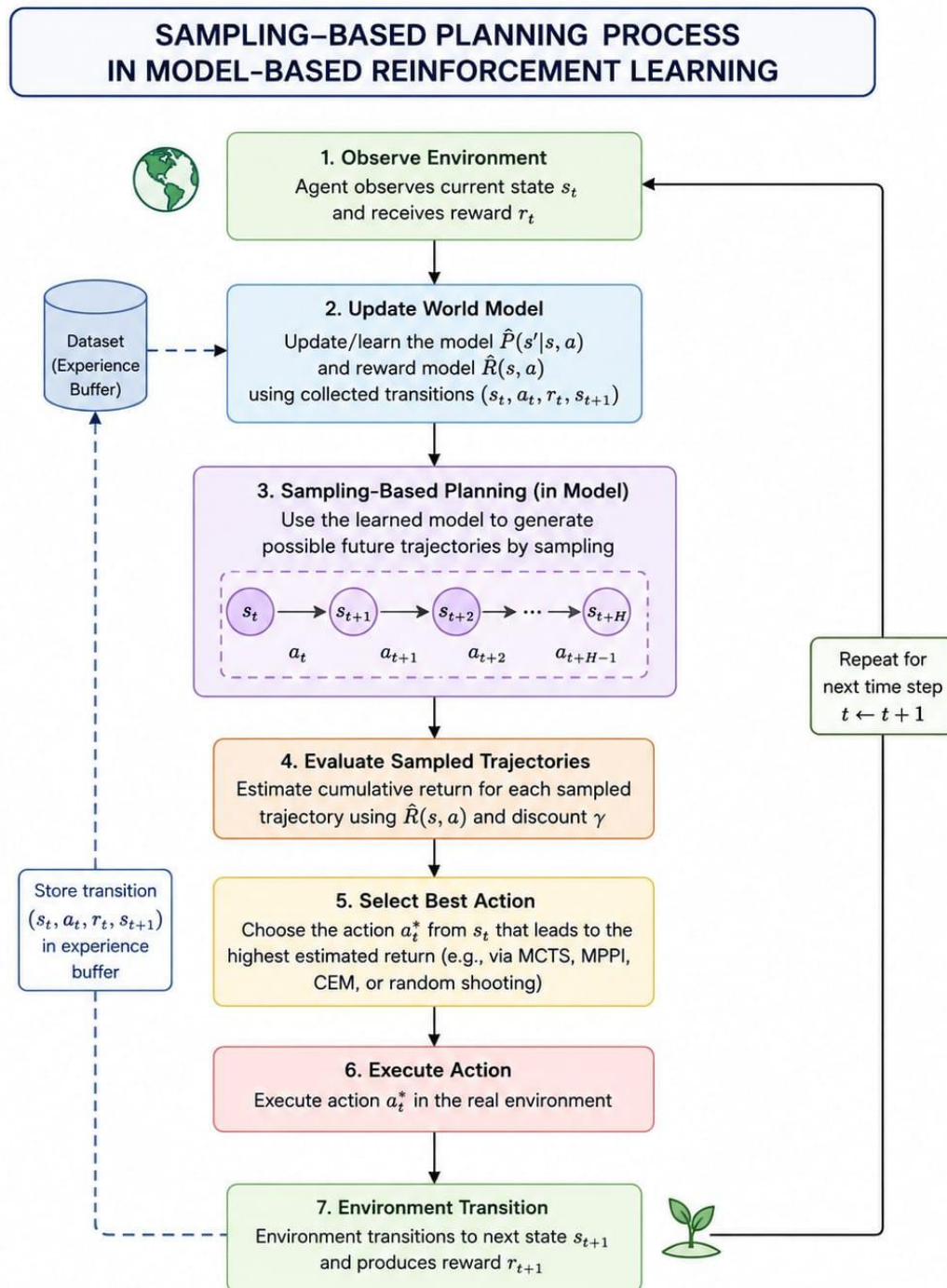
9) Prepare a flowchart explaining gradient-based optimization using backpropagation in Reinforcement Learning.



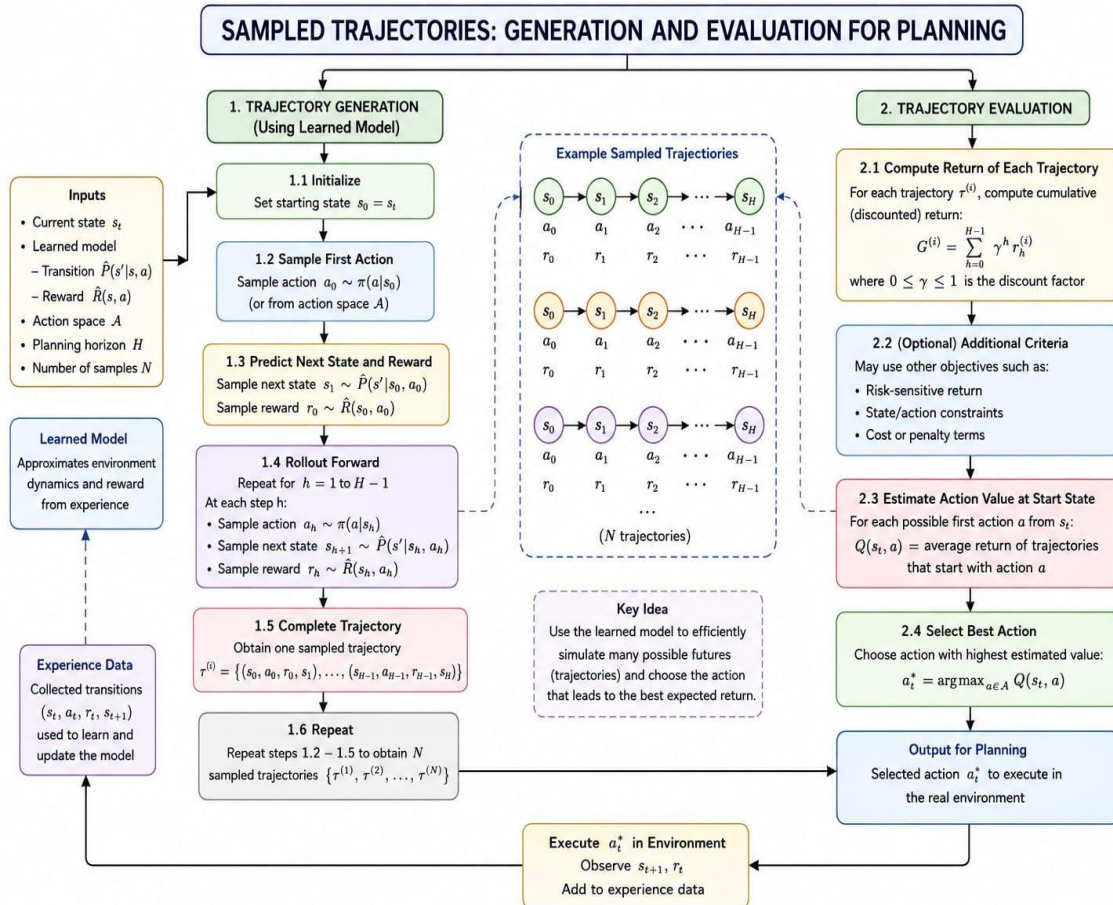
10) Draw a concept map showing the relationship among policy parameters, gradients, optimization algorithms, and rewards.



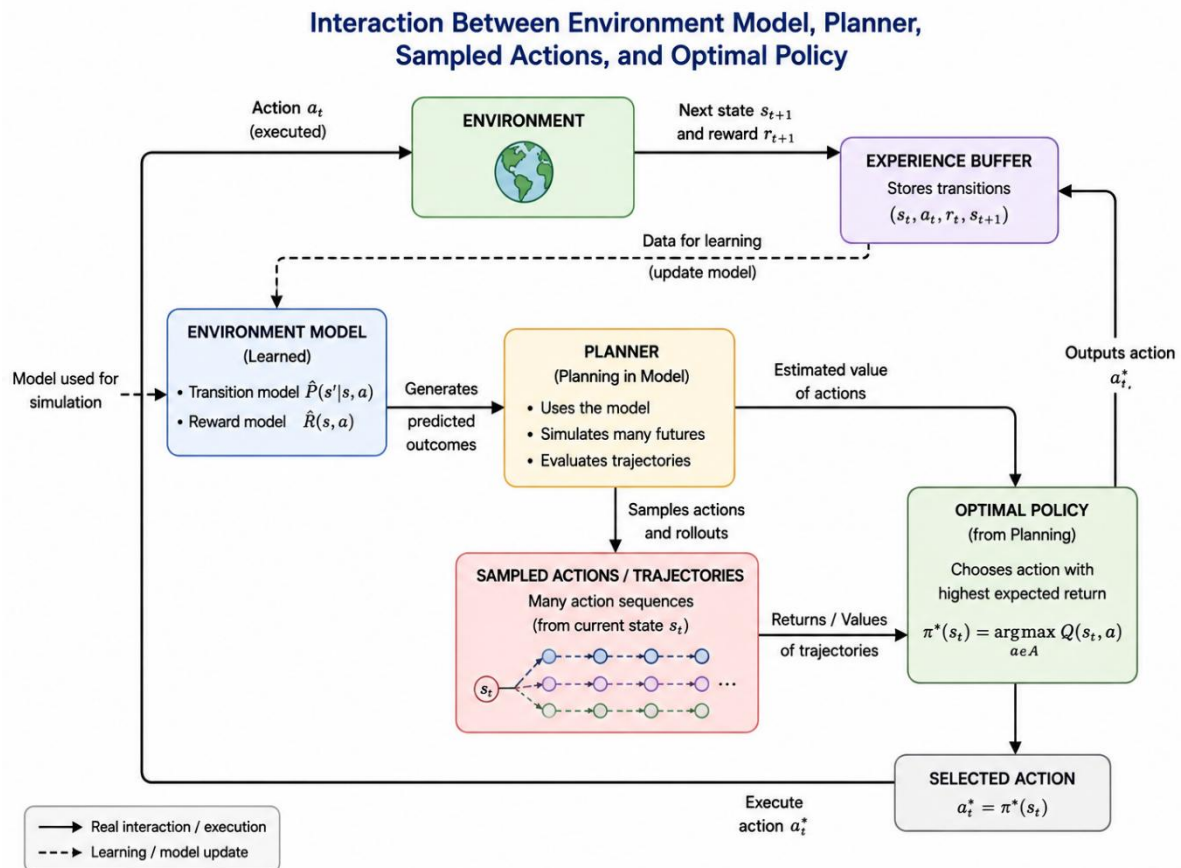
11) Draw a flowchart illustrating the sampling-based planning process in Model-Based Reinforcement Learning.



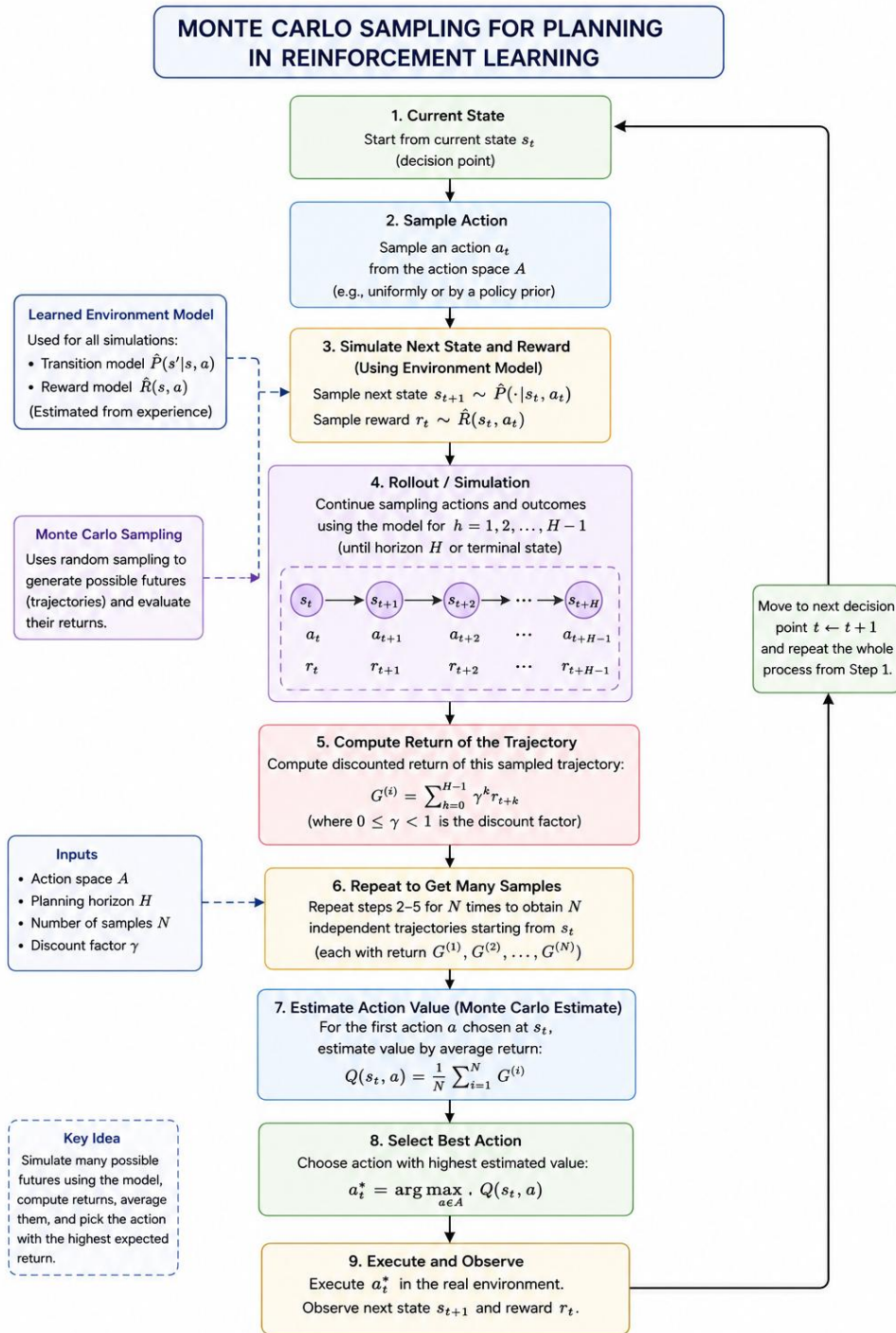
12) Prepare a concept map showing how sampled trajectories are generated and evaluated for planning.



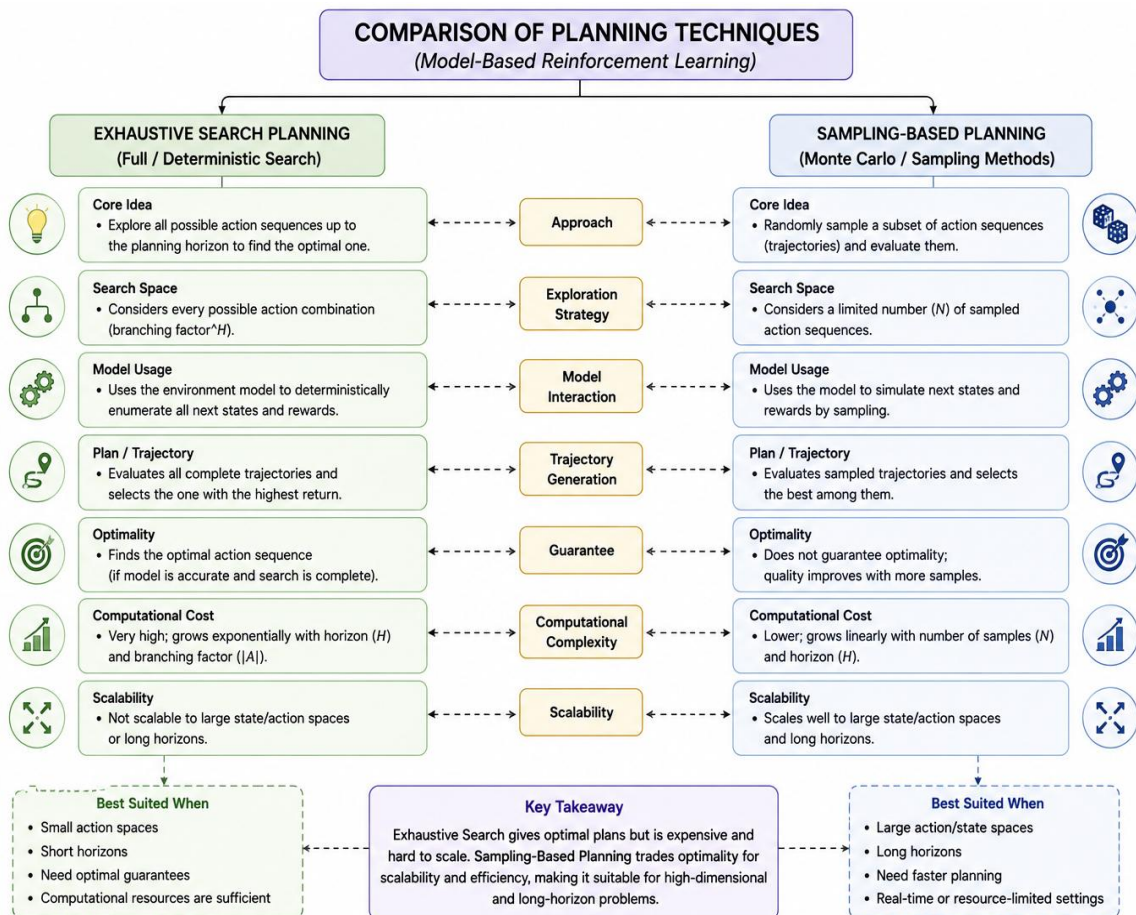
13) Draw a block diagram showing the interaction between the environment model, planner, sampled actions, and optimal policy.



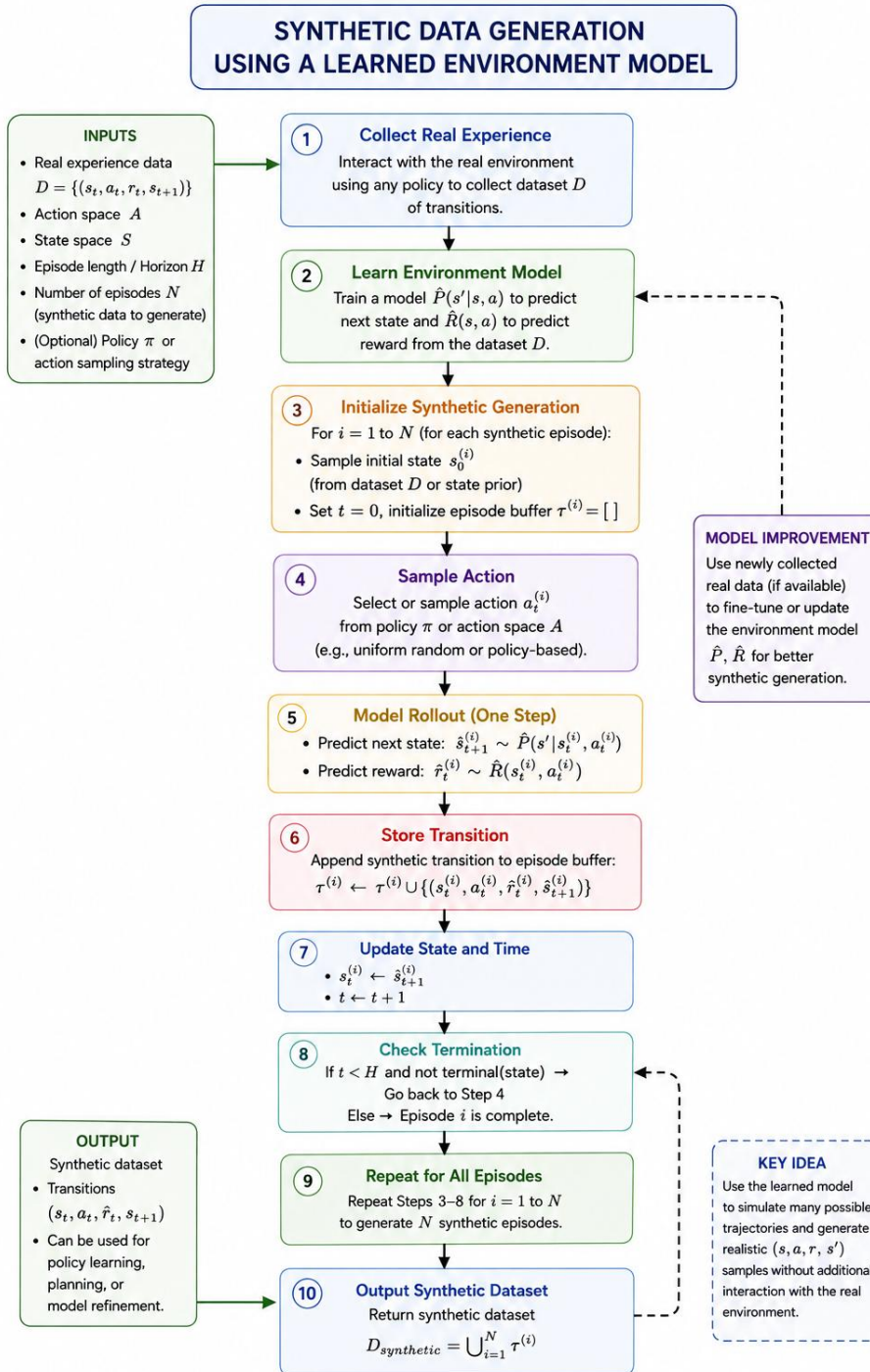
14) Design a flowchart explaining Monte Carlo sampling for planning in Reinforcement Learning.



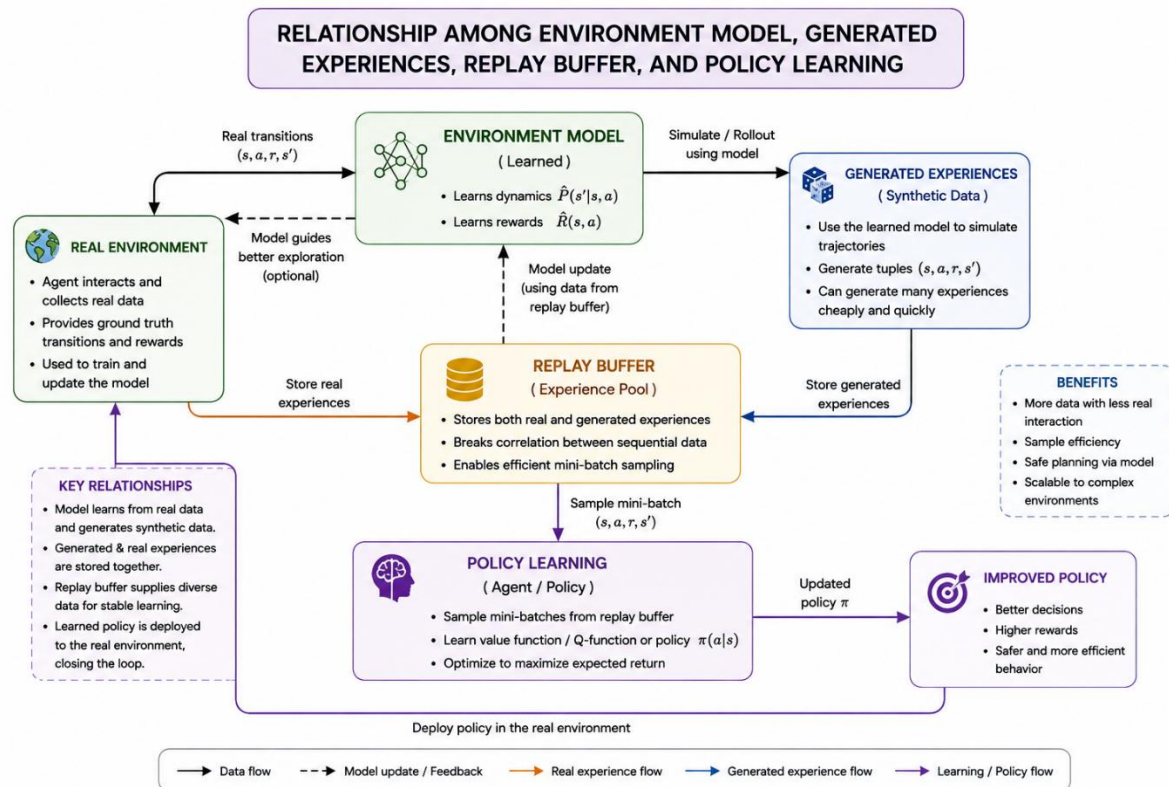
15) Draw a concept map comparing exhaustive search planning and sampling-based planning techniques.



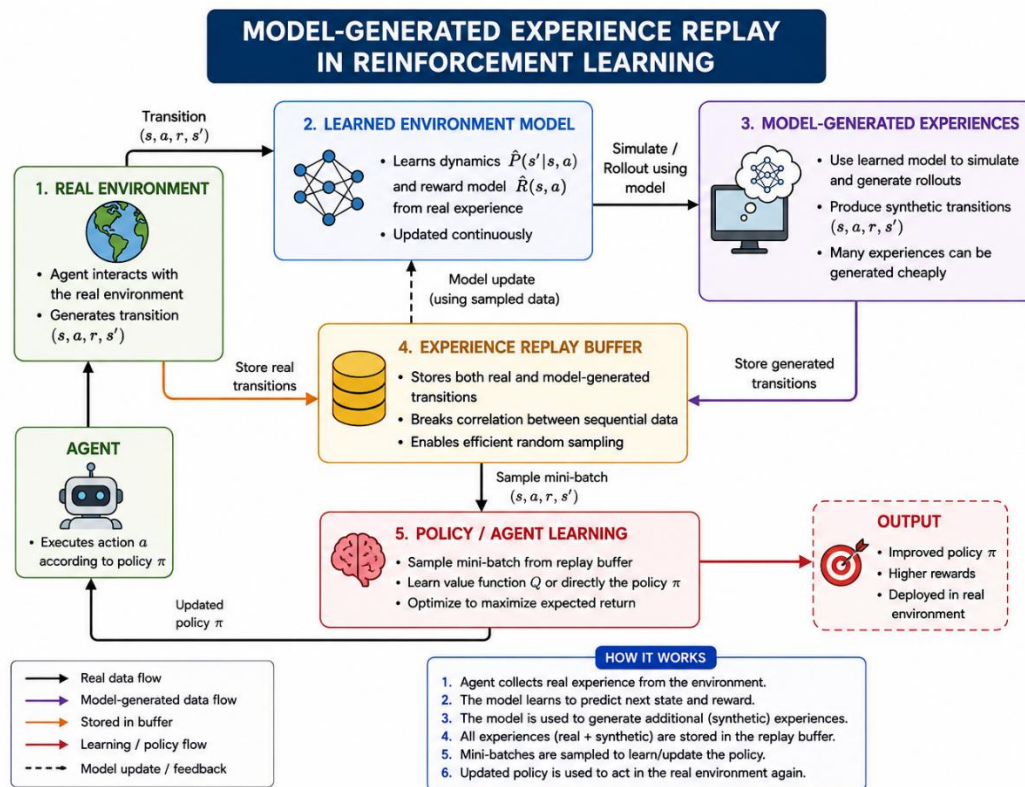
16) Draw a flowchart illustrating synthetic data generation using a learned environment model.



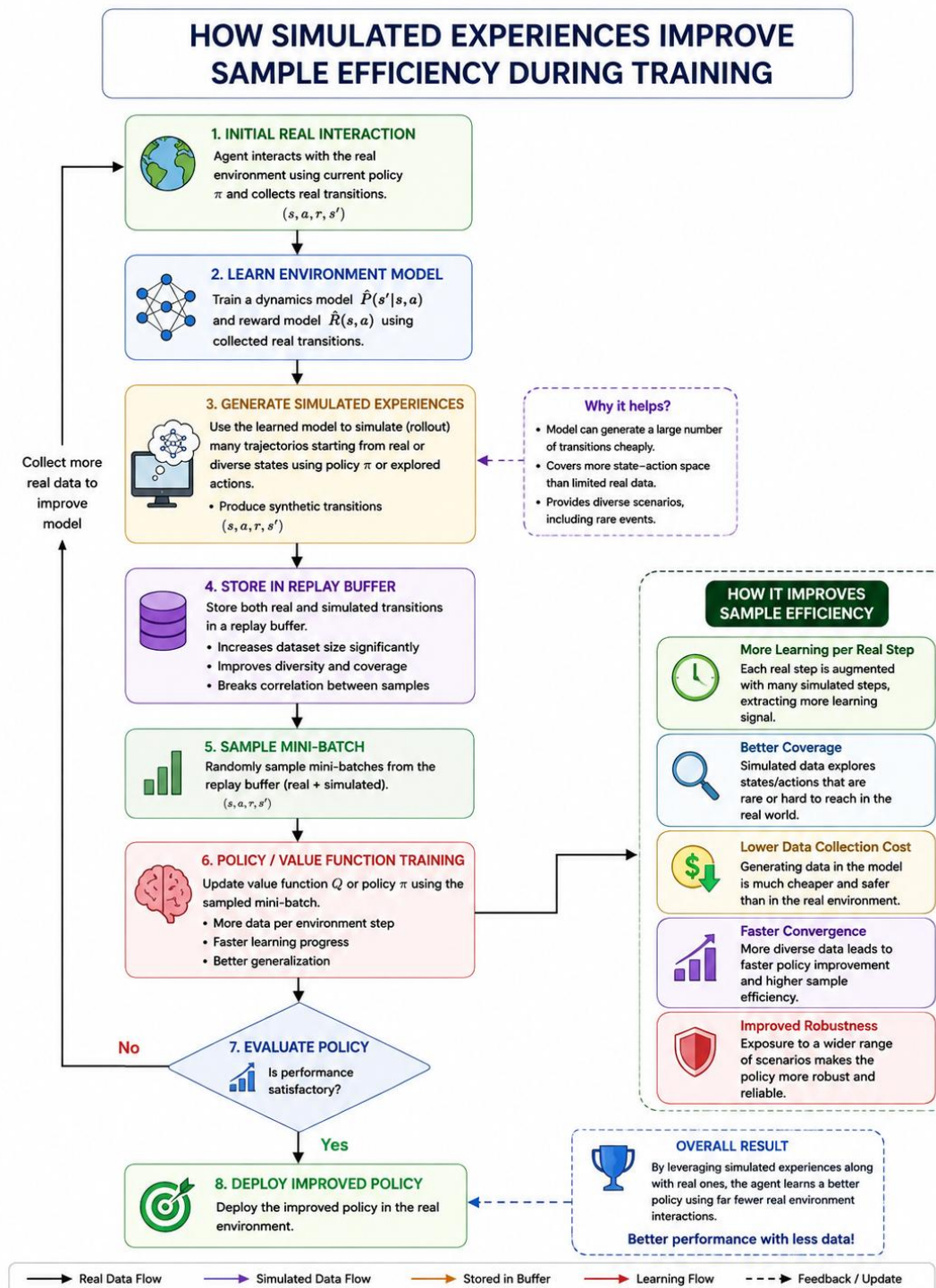
17) Prepare a concept map showing the relationship among environment model, generated experiences, replay buffer, and policy learning.



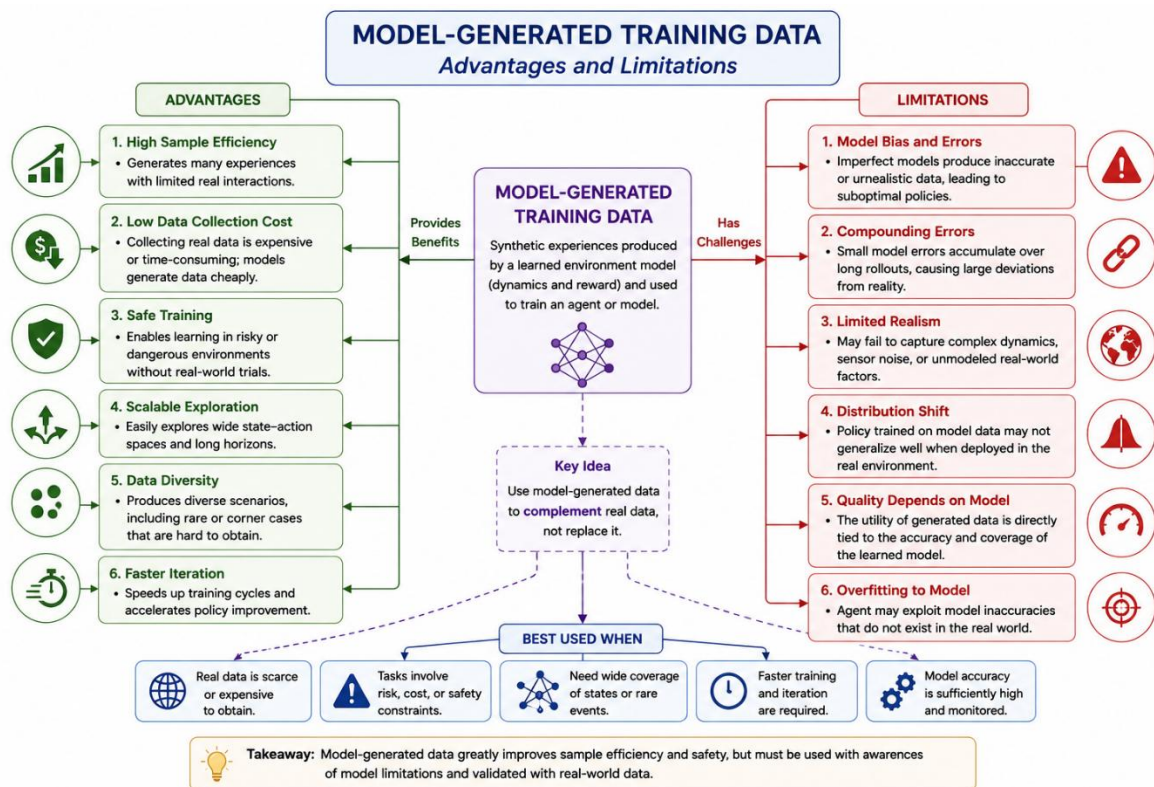
18) Draw a block diagram explaining model-generated experience replay in Reinforcement Learning.



19) Design a flowchart showing how simulated experiences improve sample efficiency during training.



20) Draw a concept map illustrating the advantages and limitations of model-generated training data.



21) Draw a flowchart explaining value-equivalence prediction for estimating future rewards.

Value-Equivalence Prediction (Estimating Future Rewards)

1 Input Current State

Given current state s_t (and action a_t if needed)

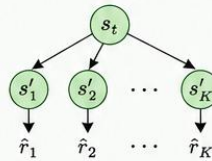


s_t

2 Predict Successor Distribution

Use learned environment model to predict possible next states and immediate rewards.

$\hat{P}(s'|s_t, a_t), \hat{R}(s_t, a_t)$



3 Compute Value-Equivalence Score

Estimate the long-term value (future reward) from each predicted next state using a value function $V(s')$.

$$EV_k = \hat{r}_k + \gamma V(s'_k)$$

4 Aggregate Expected Value

Compute the expected value over all possible next states (value-equivalence prediction).

$$\hat{V}(s_t, a_t) = \sum_{k=1}^K \hat{P}(s'_k | s_t, a_t) EV_k$$

5 Estimate State Value (optional)

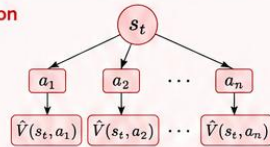
For state value, take the best action value or expected value.

$$V(s_t) = \max_a \hat{V}(s_t, a)$$

6 Use for Planning / Decision

Use the predicted value to:

- Select best action
- Evaluate rollouts
- Guide search (e.g., MCTS)
- Train policy/value networks



7 Update Value Function (Learning)

Update value function $V_\theta(s)$ (or $Q_\theta(s, a)$) to minimize prediction error.

$$L(\theta) = (V_\theta(s_t) - \hat{V}(s_t))^2$$

8 Output

Return predicted value as estimate of future rewards from the current state/action.

$$\hat{V}(s_t) \text{ or } \hat{V}(s_t, a_t)$$

What is Value-Equivalence?

Two different future trajectories (or next states) are considered value-equivalent if they yield the same expected long-term reward. We predict this expected value to estimate future rewards without full rollouts.

Key Ideas

- Use a model to look one step ahead.
- Estimate future value from each possible next state.
- Weight by transition probabilities.
- The result approximates the expected return.

Benefits

- ✓ Reduces need for long rollouts
- ✓ Faster planning
- ✓ Works well with learned models
- ✓ Improves sample efficiency

Notation

s_t : current state
 a_t : action
 s'_k : k-th possible next state
 $\hat{P}(s'_k | s_t, a_t)$: predicted transition probability
 $\hat{r}_k$: predicted immediate reward
 $V(s)$: value of state s
 γ : discount factor

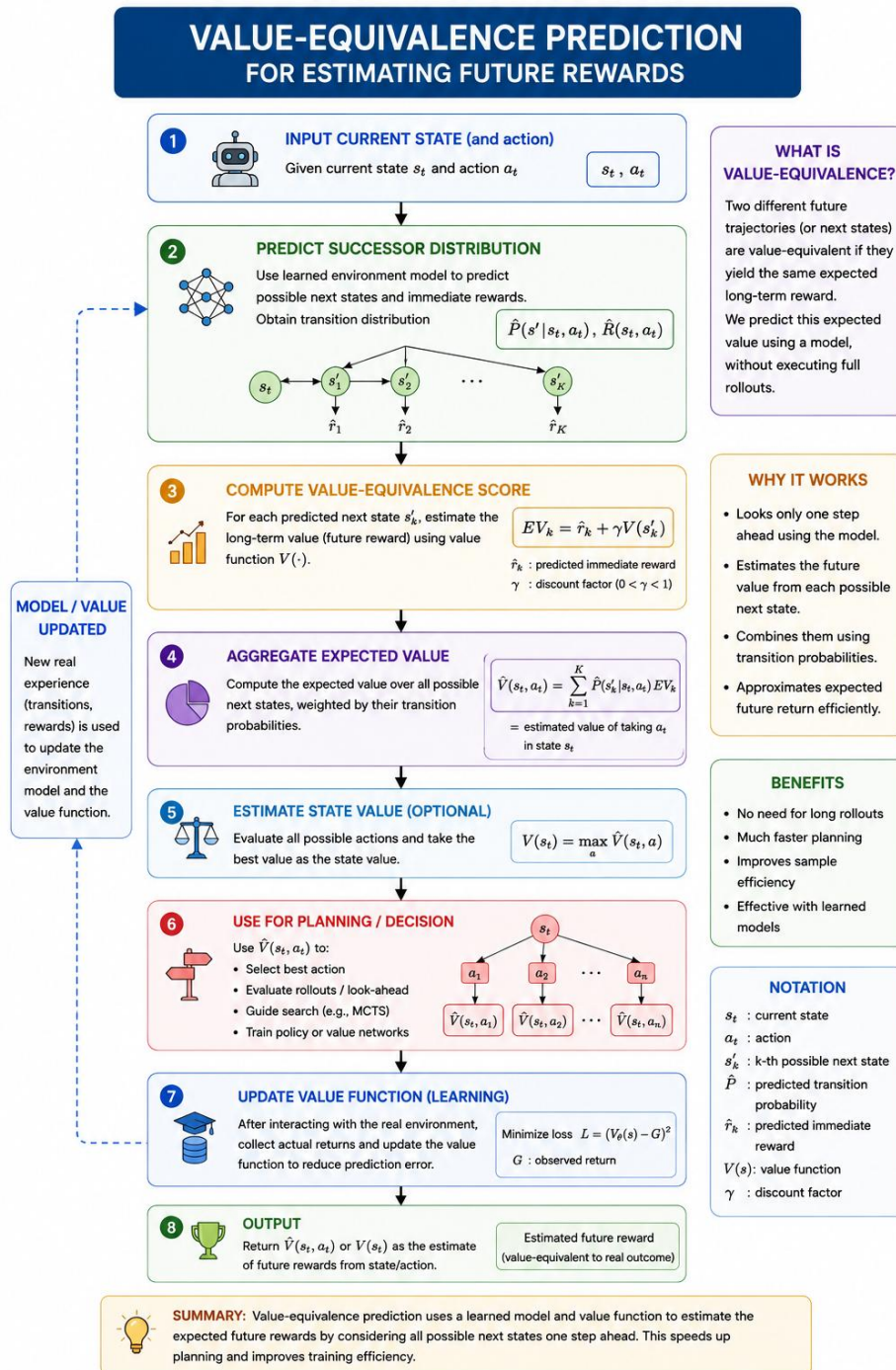
Model / Value Updated with New Data

After executing actions in the real environment, collect transitions and rewards to update the model and the value function.

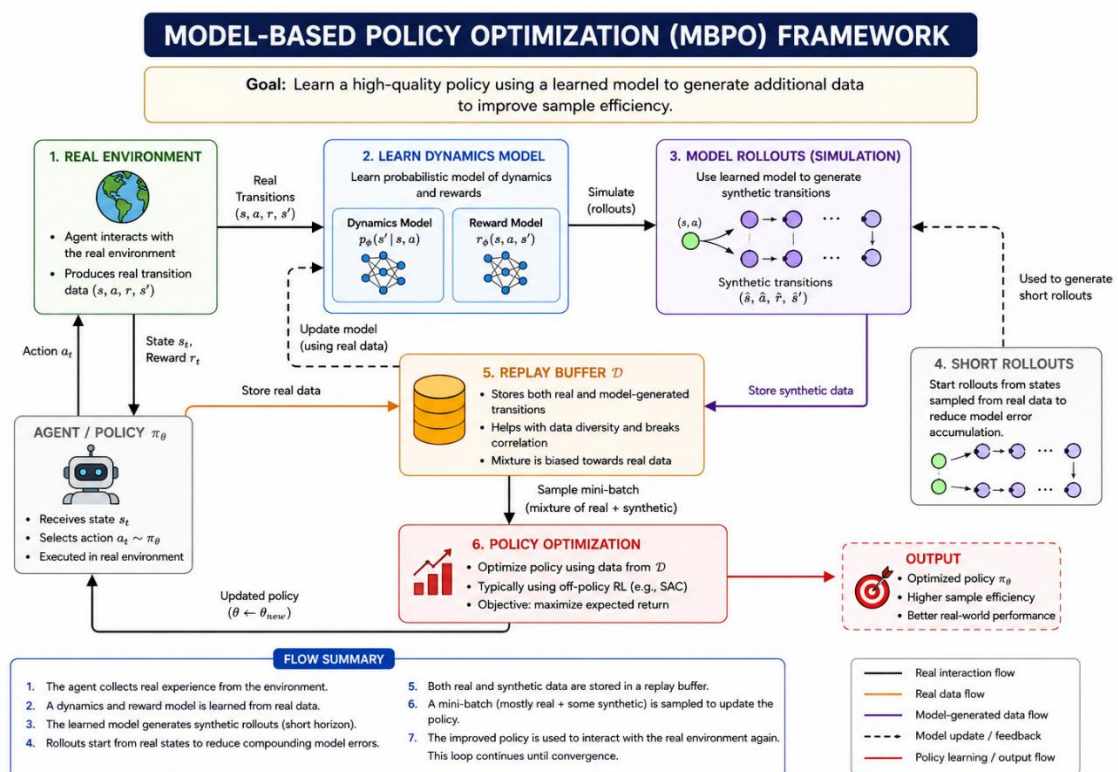


Summary: Value-equivalence prediction uses a learned model and value function to estimate the expected future rewards from a state or action by aggregating the values of possible next states. This guides better decisions and more efficient learning.

22) Prepare a concept map showing the relationship among value functions, reward prediction, policy evaluation, and policy improvement.



23) Draw a block diagram illustrating the complete Model-Based Policy Optimization (MBPO) framework.

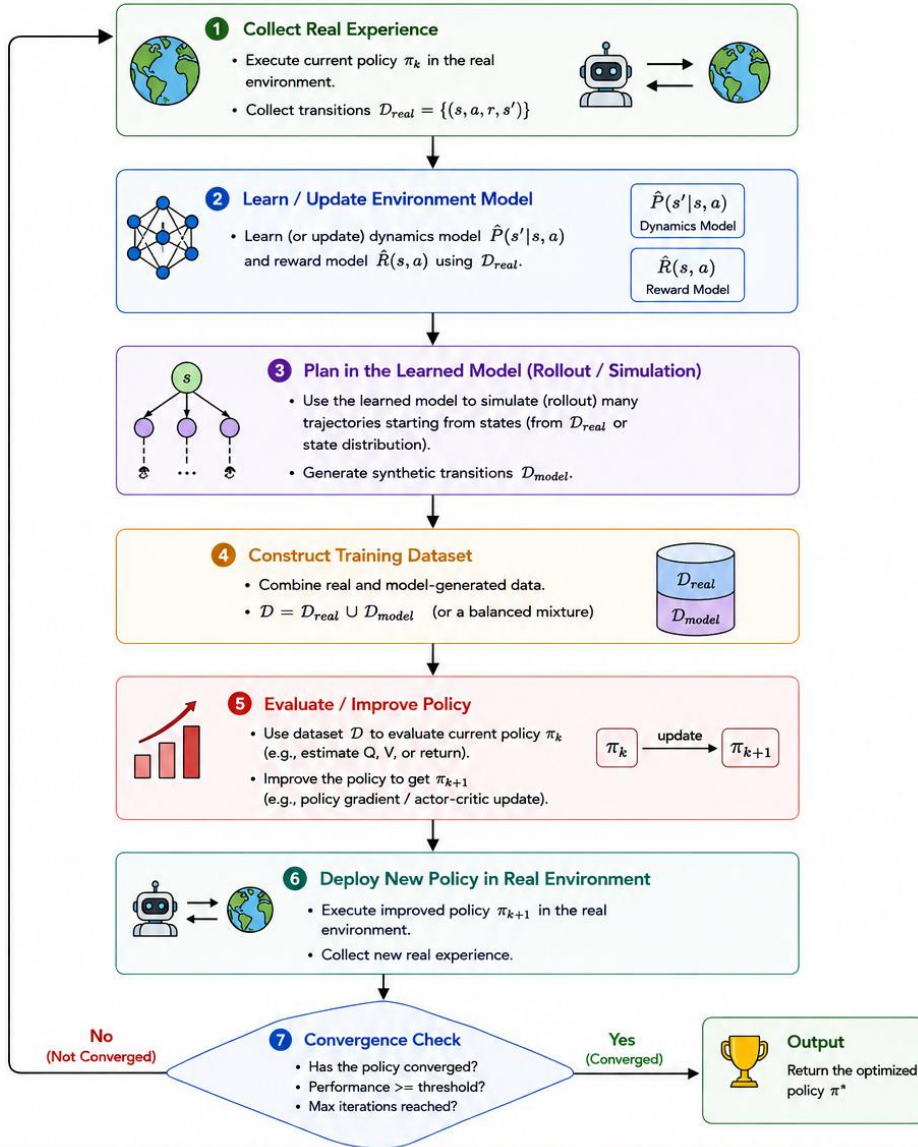


24) Design a flowchart explaining iterative policy optimization using learned environment dynamics.

ITERATIVE POLICY OPTIMIZATION USING LEARNED ENVIRONMENT DYNAMICS



Goal: Learn an optimal (or near-optimal) policy by iteratively using a learned model of the environment to improve the policy.



Iterative Loop Summary

1. Collect real data using current policy.
2. Learn / update the environment dynamics and reward.
3. Use the learned model to simulate many trajectories and generate additional data.

4. Combine real and simulated data for training.
5. Evaluate and improve the policy using this data.
6. Deploy the new policy in the real environment.
7. Repeat the loop until convergence.



Key Idea

Leverage a learned model to imagine more experience, improving sample efficiency while keeping learning grounded in real data.

25) Draw a comprehensive concept map integrating Model-Based RL, Analytic Gradient Computation, Sampling-Based Planning, Model-Based Data Generation, Value-Equivalence Prediction, and Model-Based Policy Optimization into a unified Reinforcement Learning framework.

